

PrepME AI — Day 7 Notes (Delete, Update, Authorization & Frontend Notes UI)

1. Authentication vs Authorization

Very important backend concept.

Many developers confuse these two.

Authentication

Question:

Who are you?

Examples:

Login
Password Verification
JWT Token

Authentication proves identity.

Example:

I am Shweta.

Authorization

Question:

Are you allowed to do this?

Examples:

```
Delete Note
Edit Note
View Private Data
```

Authorization checks permissions.

Example:

```
Can Shweta modify this note?
```

2. Security Problem

Suppose:

User A

```
{
  "_id": "111"
}
```

owns:

```
{
  "_id": "AAA",
  "title": "JWT Notes",
  "user": "111"
}
```

User B

```
{
  "_id": "222"
}
```

tries:

```
DELETE /notes/AAA
```

Without authorization checks:

User B can delete User A's note.

This is a serious security issue.

3. Ownership Validation

Before deleting or updating:

```
if (  
  note.user.toString()  
  !==  
  req.user.userId  
)
```

Purpose:

Check whether
the current user
owns the note.

Example

Logged-in User:

222

Note Owner:

111

Comparison:

```
111 !== 222
```

Result:

```
{  
  "message": "Not authorized"  
}
```

Action blocked.

4. Delete Note API

Created:

```
DELETE /api/users/notes/:id
```

Purpose:

```
Delete a note  
by ID
```

5. URL Parameters

Example:

```
/api/users/notes/685f123
```

Then:

```
req.params.id
```

returns:

```
685f123
```

Used to identify which note to delete.

6. New MongoDB Method

Learned:

```
findById()
```

Purpose:

```
Find a document  
using its MongoDB ID
```

Example:

```
const note =  
await Note.findById(  
  req.params.id  
)
```

Return Values

If found:

```
{  
  ...  
}
```

If not found:

```
null
```

7. Delete Controller Flow

```
Receive Note ID
  ↓
Find Note
  ↓
Check Ownership
  ↓
Delete Note
  ↓
Return Success
```

8. New MongoDB Method

Learned:

```
findByIdAndDelete()
```

Purpose:

```
Delete document
using ID
```

Example:

```
await Note.findByIdAndDelete(
  req.params.id
)
```

9. Delete Route

```
router.delete(
  "/notes/:id",
  authMiddleware,
```

```
    deleteNote  
  )
```

Flow:

```
Request  
↓  
JWT Verification  
↓  
Ownership Check  
↓  
Delete Note
```

10. CRUD Status After Delete

CREATE	✓
READ	✓
UPDATE	✗
DELETE	✓

11. Update Note API

Created:

```
PUT /api/users/notes/:id
```

Purpose:

Edit existing note

Example

Before:

```
{
  "title": "Learn JWT"
}
```

After:

```
{
  "title": "Learn JWT Deeply"
}
```

12. Update Authorization

Same ownership check required.

Reason:

```
User B should not
modify User A's note.
```

Flow:

```
Find Note
↓
Check Ownership
↓
Update Note
```

13. New MongoDB Method

Learned:

```
findByIdAndUpdate()
```

Purpose:

Find document
Update document
Return document

Example:

```
await Note.findByIdAndUpdate(  
  id,  
  {  
    title:  
      req.body.title  
  }  
)
```

14. Important Option

```
{  
  new: true  
}
```

Purpose:

Return updated document

Without it:

MongoDB returns old document

With it:

MongoDB returns updated document

Example

Before:

```
{
  "title": "JWT"
}
```

After:

```
{
  "title": "JWT Deeply"
}
```

Returned response:

```
{
  "title": "JWT Deeply"
}
```

15. Update Route

```
router.put(
  "/notes/:id",
  authMiddleware,
  updateNote
)
```

16. CRUD Completed

At this point:

```
CREATE ☒
READ ☒
UPDATE ☒
DELETE ☒
```

First complete CRUD backend finished.

17. Important Debugging Lesson

Problem encountered:

500 Internal Server Error

Cause:

Postman Body Type
was set to Text
instead of JSON

Wrong

Body
↓
raw
↓
Text

Correct

Body
↓
raw
↓
JSON

Lesson

Before blaming code always verify:

URL
Method
Headers
Body
Database Data

Many bugs come from request formatting.

18. Frontend Integration Started

Until now:

Notes CRUD

only worked in:

Postman

Now we started moving Notes into React.

19. New React State

Added:

```
const [title, setTitle]
```

Purpose:

Store note input value

Added:

```
const [notes, setNotes]
```

Purpose:

Store notes array

Example:

```
[
  {
    title: "JWT"
  },
  {
    title: "React"
  }
]
```

20. Create Note Function

Frontend:

createNote()

Flow:

```
Input
↓
Axios POST
↓
Backend
↓
MongoDB
↓
Refresh Notes
```

21. Get Notes Function

Frontend:

```
getNotes()
```

Flow:

```
JWT Token
↓
Axios GET
↓
Backend
↓
MongoDB
↓
Notes Array
↓
React State
```

22. React Rendering Lists

New React concept:

```
notes.map(...)
```

Purpose:

```
Convert array
into UI elements
```

Example:

```
[
  {
    title: "JWT"
  },
  {
    title: "React"
  }
]
```

becomes:

JWT

React

on screen.

23. Why map() Is Important

React uses:

```
array.map()
```

everywhere.

Examples:

```
Products  
Orders  
Users  
Tasks  
Notes  
Comments
```

Most lists in React use map().

24. Token Persistence

Important observation:

After:

```
Close Frontend  
Restart Frontend
```

User was still logged in.

Why?

Because JWT is stored in:

```
localStorage
```

Example:

```
localStorage.setItem(  
  "token",  
  token  
)
```

localStorage Persists

Stopping:

```
npm run dev
```

does NOT remove:

```
localStorage
```

Browser still contains:

```
token
```

Checking Token

Browser Console:


```
localStorage.getItem(  
  "token"  
)
```

Returns:

```
JWT_TOKEN
```

Removing Token

Browser Console:

```
localStorage.removeItem(  
  "token"  
)
```

Purpose:

```
Simulate Logout
```

Important Clarification

Incorrect statement:

```
I can access notes  
without login.
```

More accurate:

```
I can access notes  
without logging in again.
```

because JWT already exists.

25. Current Application Flow

```
Signup
↓
Login
↓
JWT Generated
↓
Store JWT
↓
Create Note
↓
Get Notes
↓
Update Note
↓
Delete Note
```

26. Current Concepts Mastered

- ✓ MongoDB CRUD
- ✓ JWT Authentication
- ✓ Authorization
- ✓ Ownership Validation
- ✓ Protected Routes
- ✓ Middleware
- ✓ React State
- ✓ Axios API Calls
- ✓ React map()
- ✓ localStorage
- ✓ Frontend ↔ Backend Integration

27. Next Learning Goals

Next session:

```
Logout Feature  
Automatic Login Persistence  
Auto Fetch Notes  
Delete Button in UI  
Update Note in UI  
useEffect()
```

These will make the application behave more like a real product.

Key Takeaway

Authentication answers:

"Who are you?"

Authorization answers:

"Are you allowed to do this?"

JWT tells backend
who the user is.

Ownership validation decides
what the user can modify.